一、总线仲裁的定义

总线仲裁

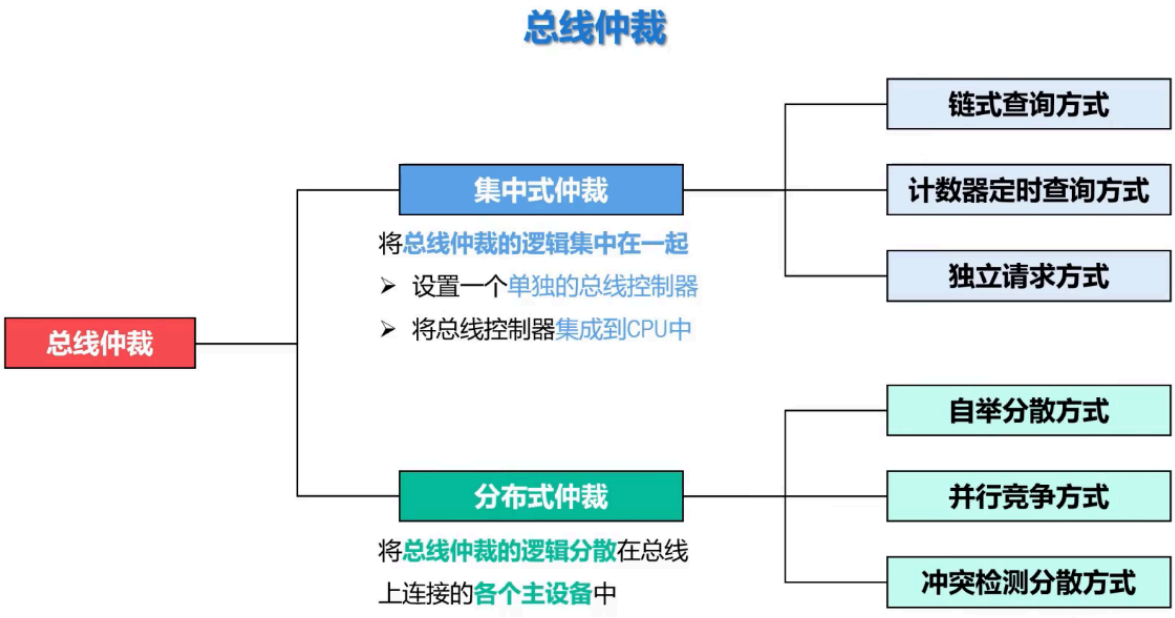
■ **总线仲裁**是计算机系统中协调多个主设备访问共享总线资源的关键机制，也称为**总线控制**或**总线裁决/判优**。

- ☐ 在**早期**计算机系统中，通常**只有CPU这一个总线主设备**，由CPU完全掌控总线的使用，**不存在共享总线的争用问题**。
- ☐ 然而，随着计算机系统结构的发展，相继出现了**多种主设备**（如DMA控制器、图形处理器GPU等），由于每个主设备都能控制总线，因此**存在共享总线的争用问题**。

■ 为了解决共享总线的争用问题，需要引入**总线控制器**，由其负责管理与分配总线的使用权。

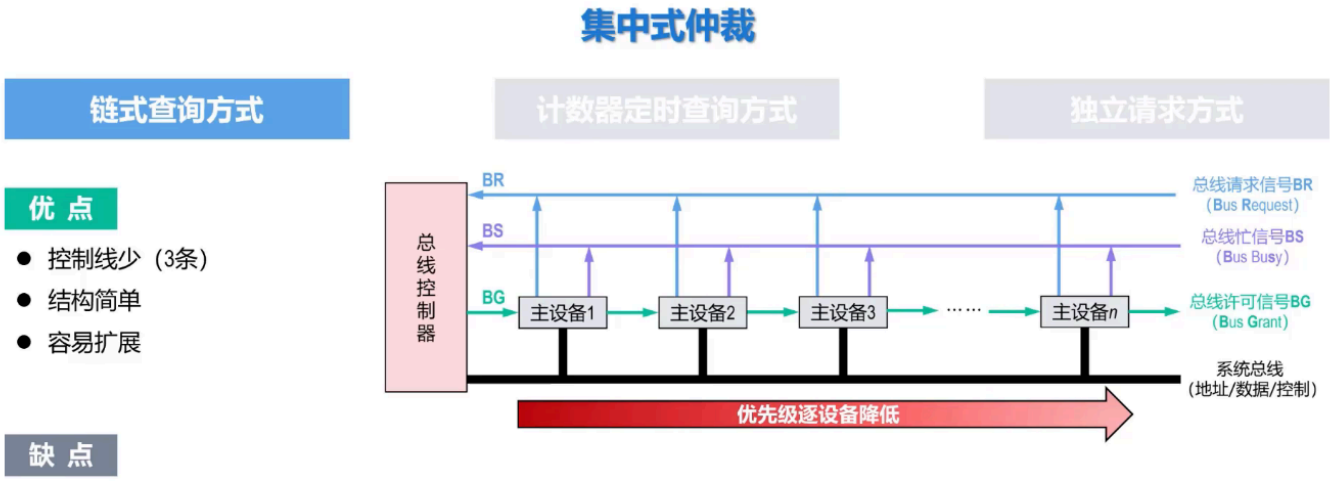
- ☐ 当总线上的多个主设备同时申请总线使用权时，由总线控制器依据一定的策略选择其中一个主设备获得总线使用权的过程，就是**总线仲裁**。
- ☐ **总线仲裁策略应考虑的问题**：
  - ➢ 各主设备的**优先级**，以确保高优先级任务及时响应。
  - ➢ 兼顾系统**公平性**，避免低优先级主设备长时间无法访问总线。
  - ➢ 为提升**系统整体效率**，仲裁过程本身应尽量快速完成，以减少总线空闲等待时间，保证数据传输的高效性。

二、总线仲裁的分类



2.1 集中式仲裁

2.1.1 链式查询



- 设备**优先级固定**（越“靠近”总线控制器的优先级越高）。当高优先级设备频繁请求使用总线时，低优先级设备长时间拿不到总线使用权（饥饿现象）。
- BG信号每向后传递1次就需要1个时钟周期，**仲裁速度慢。**
- 存在**单点故障**，菊花链中某个设备接口的链路出现故障，BG信号无法继续向后传递，造成该设备后的所有设备都不能正常工作。

仲裁过程

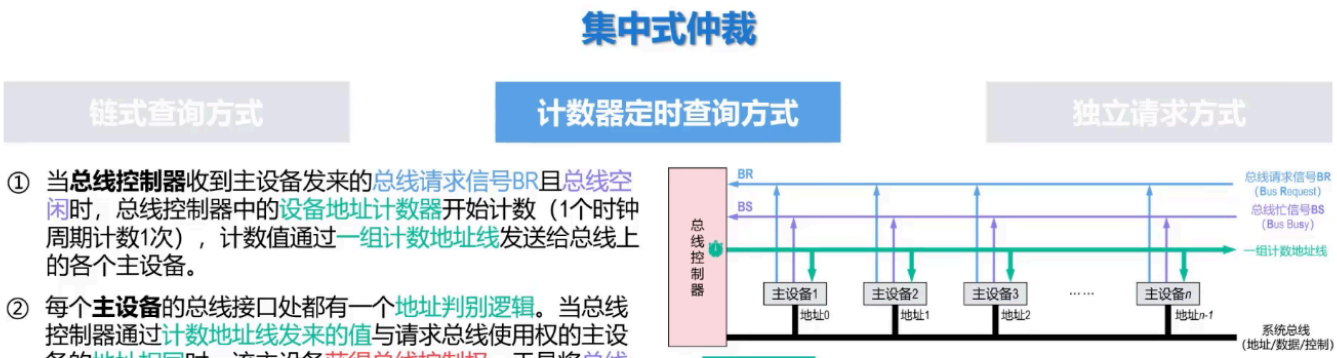
1. ==总线空闲，单设备（假设为设备2）==发出请求：

- 由于总线空闲，BS线为0
- 设备2向BR发出请求
- 请求到达总线控制器，发现BS为0，于是发出许可信号BG
- BG沿串行设备线依次传递，直至遇到第一个发出请求的设备
- 传递至设备2时，设备2获得系统总线的使用权，同时BG信号终止，BS置1

1. ==总线空闲，多设备（假设为设备1、2）==发出请求：

大致过程同上，只不过BG信号传递时，由优先级更高的设备 1 捕获，导致设备 2 无法获得权限

2.1.2 计数器定时查询



2.1.3 独立请求

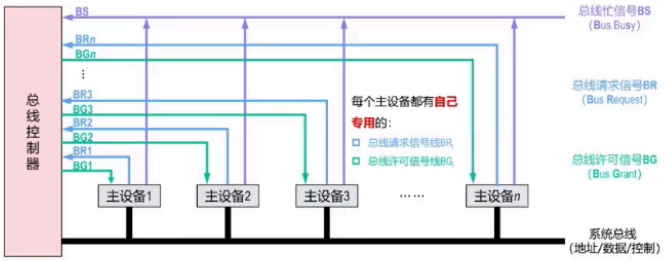
集中式仲裁

链式查询方式　　计数器定时查询方式　　**独立请求方式**

① 当**主设备**需要获取总线控制权时，通过其**专用的**总线请求信号线向总线控制器**独立**发送总线请求信号。

② **总线控制器**收到主设备发来的总线请求信号，对其进行**排队**，并**按一定优先级策略决定响应**哪个主设备的总线请求，并通过该设备**自己专用**的总线许可信号线向该设备发送总线许可信号。

③ **主设备**收到总线许可信号后就获得了总线控制权。



优 点

- **优先级策略最灵活**（固定优先级算法、公平的循环菊花链算法、FIFO和LRU等动态优先级算法）。
- 并行仲裁（BG信号不再需要逐个对设备进行串行查询），**响应时间最快。**

缺点

- **总线控制器最复杂。**
- **所需控制线数量最多**：（1+1）× *n* + 1。

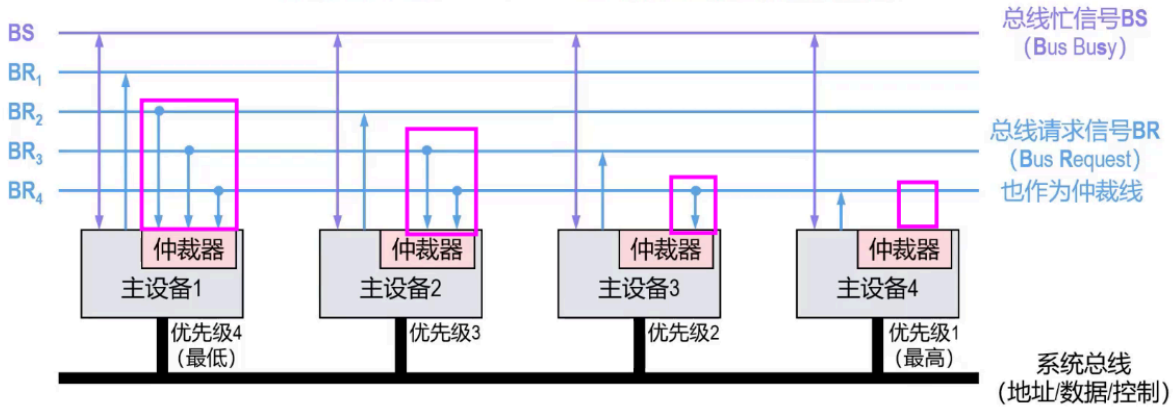
2.2 分布式仲裁

2.2.1 自举分散

分布式仲裁

自举分散方式　　并行竞争方式　　冲突检测分散方式

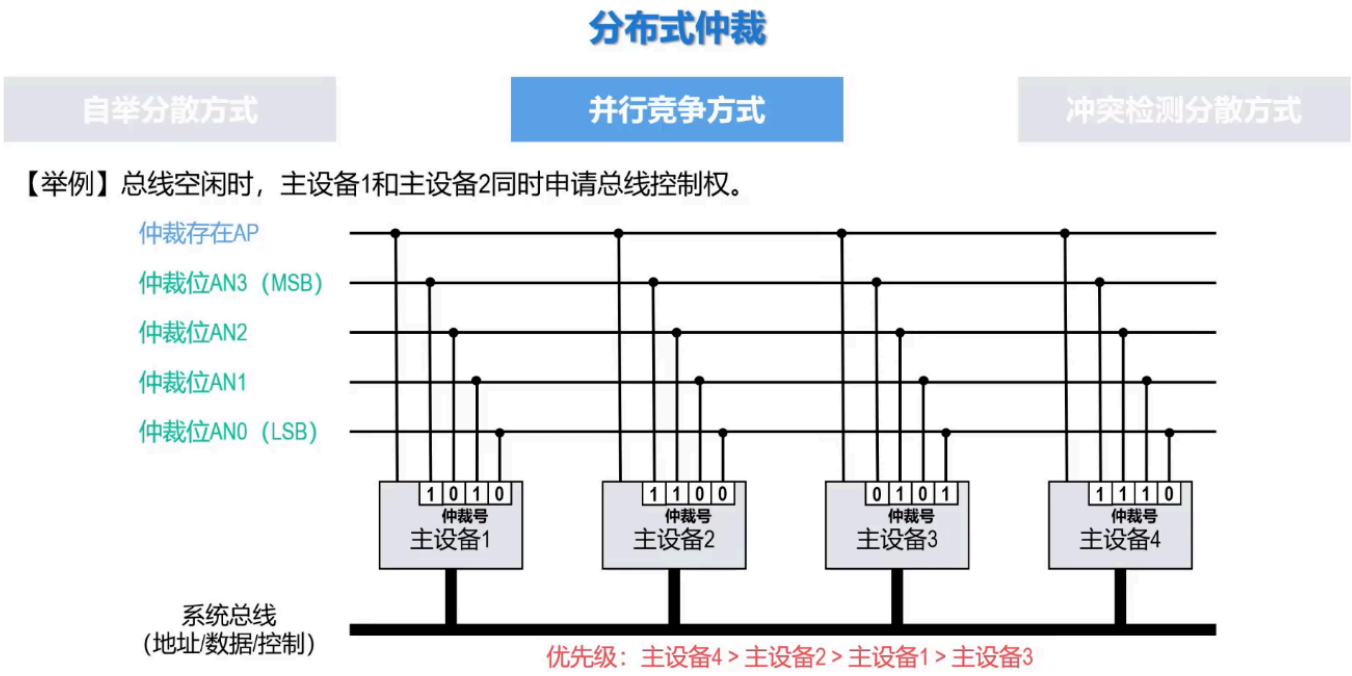
仲裁逻辑简单，但需要的总线请求和仲裁信号线较多。



注:

优先级的实现见图中粉框

2.2.2 并行竞争



==规则==

1. **每个主设备有一个唯一的4位仲裁号**（AN3~AN0），**仲裁号越大优先级越高**，所以本例中 4号(11110) > 2号(11000) > 1号(10110) > 3号(01011)
2. **仲裁线是"线或"共享的**：AP 及 AN3~AN0 是所有设备共用的信号线，只要有一个设备往某位送"1"，该线上就是"1"；每个设备既往线上发，也能从线上读回
3. **裁决规则**：各设备把自己的仲裁号放上总线，从最高位 AN3 开始逐位与线上读回值比较——**若发现自己某位发的是"0"、线上却是"1"，说明存在仲裁号更大的竞争者，立即撤销自己的仲裁号退出。**最终留在总线上的号就是获胜者的号

==逐步过程（设备1 = 10110，设备2 = 11000）==

① 提出申请：总线空闲，设备1和2同时置 AP=1（向全网广播"有仲裁在进行"），并把各自仲裁号按位送上 AN3~AN0；设备3、4 没有申请，只监听不动

② 从高位到低位逐位比较：

比较位	设备1发出	设备2发出	线上值(线或)	判定
AN3	1	1	1	双方读到 1 = 自己发的，都继续
AN2	0	**1**	**1**	设备1"发0读1"→ 有更大号 → **撤销仲裁号退出**；设备2匹配，继续
AN1	（已退出）	0	0	设备2匹配，继续
AN0	（已退出）	0	0	匹配，继续
③ **胜负在第一个不同的高位（AN2）上决出**：设备1撤下后，仲裁线上只剩设备2驱动的信号				
④ **获胜确认**：设备2读回总线上的号 = **11000 = 自己的仲裁号** → 确认获胜，取得总线控制权，开始在系统总线（地址/数据/控制）上完成传输				
⑤ **失败者善后**：设备1撤销申请后继续监听，等设备2用完总线、AP 撤销、总线回到空闲后，再重新发请求参加下一次仲裁				

==关于败者退出竞争的说明==

仲裁按固定节拍逐位推进，每一位占一个时钟周期；退出者只是从下一位起不再往线上驱动，线或值随"仍在驱动的设备集合"缩小而**自然更新**，比较照常走完 N 位。总拍数固定，与谁退出、何时退出无关

一拍之内时钟怎么安排（同步定时）

一拍（一个时钟周期）	动作
① 驱动段	尚未退出的设备把**本位值**送上对应 AN 线
② 建立段	线或电平稳定（留出建立时间：驱动延时 + 线或稳定时间，时钟周期须大于它）
③ 采样段	统一时钟沿采样：自己发的位 vs 线上值；失配者锁存"退出"标志
- AP 有效作为仲裁起点，各设备内部的位计数器对拍计数：第 1 拍比 AN3(MSB)……第 4 拍比 AN0	
- **退出决定在当拍末尾生效**：下一拍起该设备停止驱动，转入监听	
- 4 位仲裁号固定走 4 拍。	

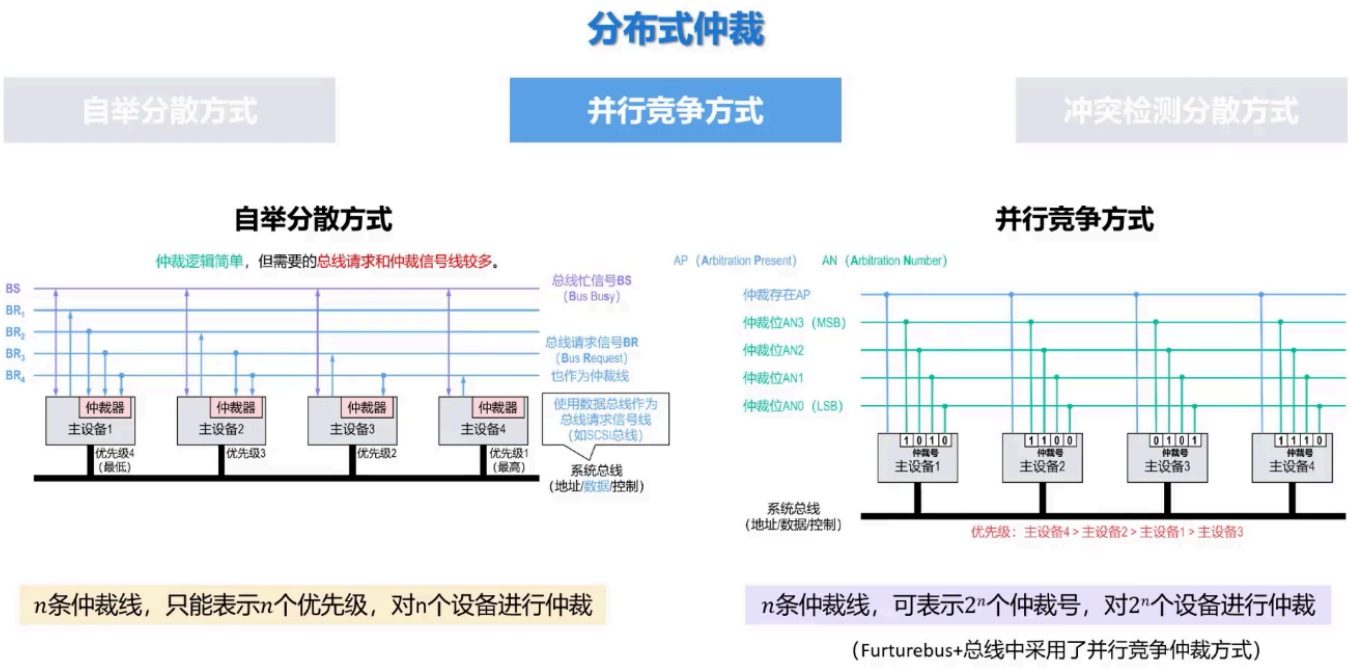
==为什么有败者退出时的当前位不用重比==

- 在第 k 位失配而退出的设备，**该位发的必是 0。**线上那个"1"是别的设备驱动的，而发 1 的设备在该位必然匹配、不会在本位退出——最终获胜者一定在这些"发 1 者"当中

- 所以退出者撤掉驱动后，**本位线上的 1 不会消失**，拍末采样值不受影响。退出者发 0 本来就近似“不驱动”状态，退出在电平上几乎没有动作
- 它的退出只影响**后续各位**的线或值——少了一个参与者，后面的“或”自动只含幸存者。这是算法的本意，不是需要修复的“脏数据”

  因此：**每一拍的线或值一经建立就不再变化**，各设备在拍末采到的结果一致可信，一次推进即可

自举分散和并行竞争的比较

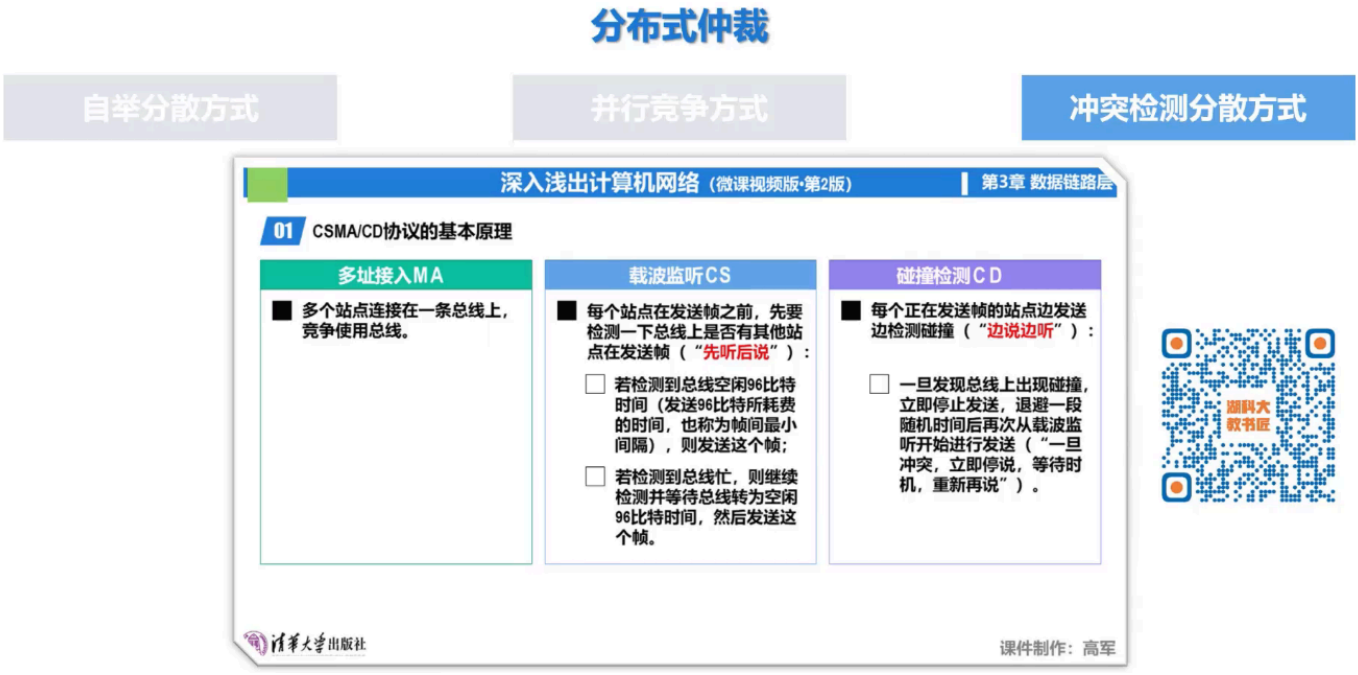


n条仲裁线，只能表示n个优先级，对n个设备进行仲裁

n条仲裁线，可表示2^n个仲裁号，对2^n个设备进行仲裁

（Furturebus+总线中采用了并行竞争仲裁方式）

2.2.3 冲突检测



见计网课程